

Desarrollo de Software

Ejercicio Guiado 2

Daniel Deblas Payo / 100522144

Xiya Ye / 100522159

25 de marzo del 2025



Índice

1. Método 1.....	2
2. Método 2.....	3
Gramática.....	3
Árbol de derivación.....	4
3. Método 3.....	5
Diagrama de flujo de calculate_balance.....	6
Diagrama de flujo de validate_iban.....	7



1. Método 1

El método 1 consiste en validar una solicitud de transferencia. Los campos que recibe esta función son dos números IBAN (`from_iban` y `to_iban`), un concepto, un tipo de transferencia, una fecha y una cantidad.

Las clases de equivalencia y los valores límite utilizados para validar cada campo de este método son los que están indicados en el apartado “*GE2 20245 FI CEVL*”. Con ellos se cubren todos los posibles valores y casos que pueden adoptar los diferentes campos.

En cuanto a los tests realizados para comprobar las clases de equivalencia válidas y valores límites válidos se han creado 5 casos de prueba que cubren todos los casos. Por otro lado, para comprobar la totalidad de las clases y valores no válidos hemos definido 32 casos de prueba que en su totalidad verifican todos los casos de clases y valores no válidos.

El resultado que devuelve nuestros tests es `TRUE` para los casos válidos o una excepción que indica el campo no válido o si ya existe una transferencia con los mismos datos.

Todos los tests utilizado se encuentran detallados en el documento excel en la página “*GE2 2024 FI*” y en el archivo python del proyecto “*test_transfer_request_tests.py*”.

2. Método 2

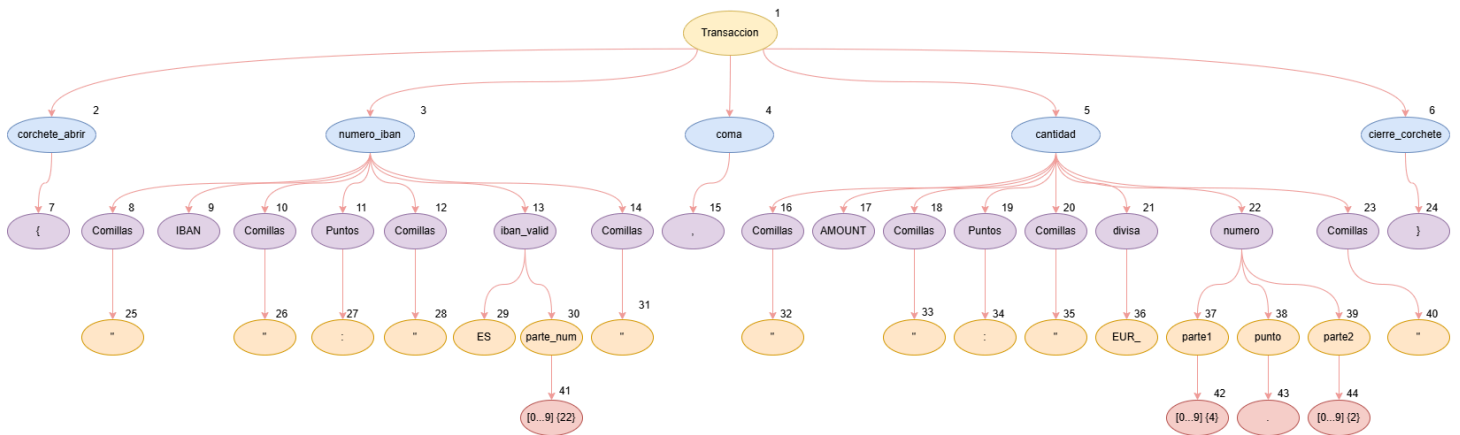
La funcionalidad del método 2 es registrar el ingreso de dinero en una cuenta. Para ello recibe un archivo JSON que debe cumplir con el formato solicitado en el enunciado. Para eso, hemos implementado la siguiente gramática.

Gramática

```
Transaccion ::= corchete_abrir numero_iban coma cantidad cierre_corchete
corchete_abrir ::= {
cierre_corchete ::= }
coma ::= ,
numero_iban ::= comillas IBAN comillas puntos comillas iban_valid comillas
comillas ::= "
puntos ::= :
iban_valid ::= ES parte_num
parte_num ::= [0...9] {22}
cantidad ::= comillas AMOUNT comillas puntos comillas divisa numero comillas
divisa ::= EUR_
numero ::= parte1 punto parte2
parte1 ::= [0...9] {1-4}
parte2 ::= [0...9] {1-2}
punto ::= .
(*) el guion bajo (_) en divisa representa un espacio en blanco
```

El árbol de derivación que hemos sacado a partir de la gramática es el siguiente.

Árbol de derivación



A partir de ese árbol de derivación hemos creado los tests que verifican todos los casos de prueba de los nodos. En total hay 47 tests. El primero de ellos verifica el caso válido. El resto, comprueban el correcto funcionamiento en los casos de borrado y duplicación de los nodos no terminales y la modificación de los nodos terminales.

Las correspondientes pruebas realizadas se encuentran indicadas en la página excel “GE2 2024 F2” y en el archivo “test_deposit_into_account.py”

3. Método 3

La funcionalidad del método 3 consiste en calcular el saldo de una cuenta a partir de transacciones recibidas con el formato: {IBAN: "", amount:"+2483,43"}. Para ello, el método valida que el IBAN recibido es válido. En caso de que sea válido va a devolver un fichero de tipo json donde se guardarán los datos de la transacción. En caso de que el IBAN recibido no sea válido, devolverá una excepción. También se mostrará una excepción en el caso de que falte el fichero con las transacciones o que haya un error de procesamiento interno. Para validar el IBAN, hemos hecho uso de la función *validate_iban* ya desarrollada con anterioridad.

Para verificar la validez de nuestro método, hemos implementado unos tests para probar que funciona correctamente. Primero hemos dibujado los diagramas de flujo de los métodos usados (*calculate_balance* y *validate_iban*) para posteriormente decidir nuestros caminos básicos y a partir de ahí sacar los todos los caminos posibles para validar nuestros métodos. Aparte de estos caminos, como tenemos un bucle en *calculate_balance*, hemos añadido algunos casos para comprobar que nuestro método ejecuta correctamente los bucles y entra en el bucle las veces que sean. Hemos tenido en cuenta 5 casos para los bucles: que no entre, entra una vez, entra 2 veces, entra 99 veces y entra 100 veces.

A la hora de implementar los tests en python, no hemos podido probar algunos caminos, debido a que no eran posibles de producir, como el test 17, ya que en el nodo 12 la variable `found = True`, pero en el nodo 9 dice que `not found` es `True`, por lo que no es posible.

Todos los tests están en el excel con más detalles en el excel en la página “GE2 2024 F3”.

Diagrama de flujo de calculate_balance

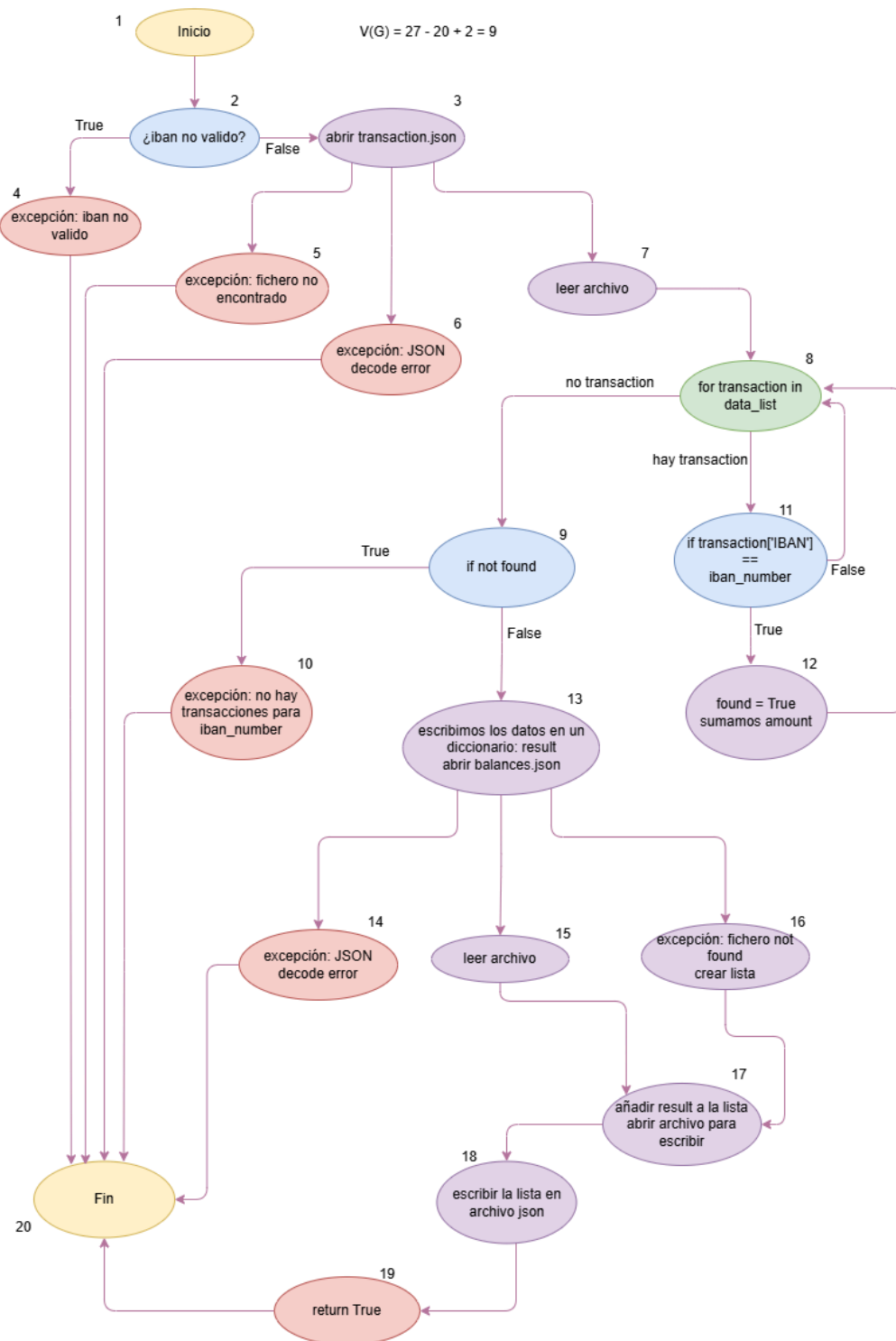


Diagrama de flujo de validate_iban

